

**A RESILIENT AND TRUST-BASED OTA
FRAMEWORK FOR IOT DEVICES :
SECURETRUSTOTA**

Project ID : 25-26j-058
Project Proposal Report

IT22365996
Mandira U.L.L.K

BSc. (Hons) in Information Technology Specializing in Cybersecurity

Department of Computer Systems Engineering

Faculty of Computing

Sri Lanka Institute of Information Technology Sri Lanka

August 2025

A RESILIENT AND TRUST-BASED OTA FRAMEWORK FOR IOT DEVICES : SECURETRUSTOTA

Project ID : 25-26j-058
Project Proposal Report

IT22365996
Mandira U.L.L.K

Supervisor : Mr. Kavinga Yapa Abewardena
Co supervisor : Ms. Dinithi Pandithage

BSc. (Hons) in Information Technology Specializing in Cybersecurity

Department of Computer Systems Engineering

Faculty of Computing

Sri Lanka Institute of Information Technology Sri Lanka

August 2025

DECLARATION

I declare that this is my own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature	Date
Mandira U.L.L.K	IT22365996		19.09.2025

Name of the supervisor: Mr. Kavinga Yapa Abeywardena

Name of co-supervisor: Ms. Dinithi Pandithage

The above candidate has carried out research for the bachelor's degree dissertation under my supervisor.

Signature:
Supervisor: Mr. Kavinga Yapa Abeywardena

Signature:
Co-Supervisor: Ms. Dinithi Pandithage

Data : 18.09.2025

ABSTRACT

The rapid expansion of the Internet of Things has introduced a pressing need for secure, efficient, and reliable mechanisms to deliver over the air firmware updates to millions of connected devices. Traditional update methods, while effective for powerful computing platforms, are often unsuitable for resource constrained IoT devices due to limited memory, bandwidth restrictions, and energy constraints. This research proposes a lightweight OTA update framework specifically designed for microcontroller class devices such as the ESP32, integrating strong cryptography, bandwidth optimization, and fail safe mechanisms. The methodology involves combining elliptic curve cryptography and AES-CCM encryption to ensure confidentiality, integrity, and authenticity of firmware updates without imposing excessive computational overhead. Delta update techniques such as bsdiff and xdelta3 will be employed to transmit only binary differences between firmware versions, reducing network load by up to 90% compared to full transfers. Anticipated results include significant bandwidth savings, enhanced security without performance degradation, and improved reliability in heterogeneous IoT environments. By addressing key limitations in existing solutions, the research aims to demonstrate that secure and efficient OTA is achievable even for devices with minimal resources. The conclusions are expected to emphasize the feasibility of lightweight OTA as a scalable solution for smart homes, industrial systems, and remote sensor networks. Recommendations will focus on adapting the proposed framework for diverse IoT applications and integrating it with existing cybersecurity standards to support long term device sustainability.

Keywords: *Secure OTA, IoT Security, Delta Updates, ECC, AES-CCM, Lightweight Cryptography*

ACKNOWLEDGMENT

I would like to sincerely express my gratitude to supervisors, Mr. Kavinga Yapa Abeywardena and Ms. Dinithi Pandithage, Department of Computer Systems Engineering at the Sri Lanka Institute of Information Technology, for their constant support and guidance regarding the development of this research proposal. Their knowledge and constructive feedback have been invaluable in shaping the direction of this project and in addressing possible challenges.

I would also like to thank to all those who have in one way or the other participated in the accomplishment of this study. Lastly, my appreciation goes to the coordinators and the design and analysis lecturer of this course for all round support during the whole proposal phase. Their guidance has been fundamental in laying a strong foundation for this project.

TABLE OF CONTENTS

Contents

DECLARATION	3
ABSTRACT.....	4
ACKNOWLEDGMENT.....	5
TABLE OF CONTENTS.....	6
LIST OF FIGURES	8
LIST OF TABLES	8
LIST OF ABBREVIATIONS.....	9
1. INTRODUCTION	10
1.1 Research Background	10
1.2 Research Scope	12
1.3 Background and Literature Survey	13
1.3.1 Overview of OTA Security in IoT	13
1.3.2 Delta Updates and Bandwidth Optimization.....	13
1.3.3 Lightweight Cryptography for Constrained Devices	14
1.3.4 Lightweight OTA Efficiency Enhancements	14
1.3.5 Challenges and Limitations.....	14
1.3.6 Conclusion	15
1.4 Research Area	16
1.4.1 IoT Security Engineering	16
1.4.2 Embedded Systems Development.....	16
1.4.3 Cryptography for Constrained Devices.....	16
1.4.4 Wireless Communication Protocols	16
1.5 Significance of the Research.....	17
2. RESEARCH GAP	18
3. RESEARCH PROBLEM.....	20
3.1 Research questions.....	20
4. AIM AND OBJECTIVES.....	21
4.1 Main Objective.....	21
4.2 Specific Objective	21
5. RESEARCH METHODOLOGY	23
5.1 Overview of Methodology	23
5.2 Requirement Analysis	23

5.3	Technology Selection.....	24
5.4	System Diagram.....	25
5.5	Component Diagram.....	27
5.6	Dataset and Testing Environment.....	29
5.7	Development Flow.....	31
5.8	Evaluation Plan.....	31
5.8.1	Baselines.....	31
5.8.2	Independent Variables.....	31
5.8.3	Dependent Variables and Metrics.....	31
5.8.3	Equipment and Tools.....	32
5.8.4	Acceptance Criteria.....	32
5.8.5	Experimental Steps.....	32
6.	SELECTION OF TOOLS AND TECHNOLOGY.....	33
6.1	Technologies.....	33
6.2	Tools.....	34
7.	PROJECT REQUIREMENTS.....	36
7.1	Functional Requirements.....	36
7.2	Non Functional Requirements.....	36
7.3	User Requirements.....	37
7.4	System Requirements.....	38
8.	BUDGET AND BUDGET JUSTIFICATION.....	39
8.1	Total Estimated Budget.....	39
9.	GANTT CHART.....	42
10.	COMMERCIALIZATION.....	43
11.	CONCLUSION.....	45
	REFERENCES.....	46

LIST OF FIGURES

Figure 1: System diagram.....24

Figure 2: Component diagram.....26

Figure 3: Datasets.....28

Figure 4: Datasets.....28

Figure 5: Datasets.....29

Figure 6: Datasets.....29

Figure 7: Gantt Chart.....41

LIST OF TABLES

Table 1. List of Abbreviations.....9

Table 2. Research Gap Table.....18

Table 3. Experimental Steps.....31

Table 4. Selected tools and descriptions.....33

Table 5. Budget Summary Table.....38

LIST OF ABBREVIATIONS

Abbreviation	Definition
OTA	Over The Air
IoT	Internet of Things
ECC	Elliptic Curve Cryptography
AES - CCM	Advanced Encryption Standard Counter with CBC MAC
ESP32	Espressif Systems 32 bit Microcontroller
API	Application Programming Interface
VPS	Virtual Private Server
IDF	IoT Development Framework (ESP-IDF)
GUI	Graphical User Interface
CI/CD	Continuous Integration / Continuous Deployment
MQTT	Message Queuing Telemetry Transport
NLP	Natural Language Processing
TLS	Transport Layer Security

Table 1. List of Abbreviations

1. INTRODUCTION

1.1 Research Background

The Internet of Things is now one of the most innovative technological frameworks of the 21st century, linking together billions of devices, from home appliances to large industrial machines. It provides important functions in healthcare, logistics, education, smart cities, agriculture, and more, making it an essential part of the digital infrastructure of the era. In addition, as such an ecosystem is developing, the ability to remotely change device software has become equally important in device security and software defect maintenance. Over the Air updates serve as efficient methods to electronically modify firmware and purge unnecessary maintenance expenses.

However, such convenience comes along with the OTA security issues. OTA systems are quite easy to attack and, without appropriate safeguards, evildoers may intercept, modify, or swap firmware packages with an infected one containing malicious code. The consequences of unsecured IoT devices have already been witnessed with the Mirai botnet attack. In particular, compromised devices have been utilized to execute distributed denial of service attacks, infringe essential services, and breach sensitive user information [1].

This difficulty is further compounded by the nature of IoT equipment such as limited processing capabilities, limited memory storage and low network bandwidth. These restrictions render it unfeasible to implement the standard security solutions which are typically employed in the enterprise systems [1]. The algorithms like RSA or regular TLS protocols are very secure, but they are very expensive to run, which burns the battery of the device and slows down the transmission of the firmware. This security/performance trade off is one of the most urgent challenges to IoT firmware management.

Moreover, the variability of computers connected to IoT complicates even more the security of OTA updates. Devices are commonly distributed on various platforms, they communicate in various protocols, and they are highly dynamic. Others are located in distant or inaccessible places and there are few maintenance opportunities. With this, it is not just a technical but also a practical requirement to ensure updates are secure and lightweight in such cases.

With the ever changing cyber threats to IoT devices, there has been an immediate need to find lightweight, but strong OTA security solutions. Recent studies in this area of work are paying more attention to how efficient cryptographic algorithms could be integrated together with optimization approaches adapted to resource constrained systems. Lightweight cryptographic implementations like Elliptic Curve Cryptography and authenticated encryption modes like AES-CCM have been found as attractive options to achieve a trade off between high level of security and low resource usage. Together with the use of techniques like delta updates, in which only the altered content of firmware is transmitted, the overall effectiveness of OTA systems can be improved by a significant margin [2]. Additionally, this project explores applying lightweight natural language processing techniques to analyze firmware change patterns and metadata to guide delta generation and compression, enabling more intelligent patch segmentation and further reducing transmission size. Overall, this Lightweight Secure OTA component increases update efficiency by integrating lightweight cryptography, delta optimizations, and NLP assisted patch generation capabilities that many past OTA systems lack.

To conclude, although OTA updates are essential to maintain the IoT ecosystem, they are vulnerable in terms of security unless security issues are handled adequately. Thus, creating the mechanisms that would provide both safety and efficiency without maxing out devices capabilities is a necessary move towards the safe and scalable IoT deployment across the planet.

1.2 Research Scope

The research aims at designing as well as developing a Lightweight Secure OTA Core targeted at resource constrained IoT devices, and in this instance, the implementation of this is based on the ESP32 platform. May the ESP32 is chosen, because of built in Wi Fi connectivity, embedded hardware based cryptographic acceleration, and the embedded nature, which make it an optimal testbed to test lightweight OTA mechanisms [2].

The system proposed incorporates modern cryptography techniques where digital signature is done using Elliptic Curve Cryptography and AES-CCM is used as authenticated encryption. All these methods guarantee that data confidentiality and integrity are maintained throughout delivery of firmware, and at the same time keeping the computational overhead to a minimum. Also, a delta update system will be implemented to transfer only the binary differences between versions of the firmware, thus minimizing update sizes, saving bandwidth, and minimizing update latency. This is especially the case with the IoT devices deployed in environments where networking is scarce or expensive, and efficient transmission is a critical need [3].

The backend infrastructure will be deployed on a Virtual Private Server and it will handle the basic services such as controlling of firmware version, production of patches, encryption of updates files and safe distribution to devices. It will be written in Python, which will make these services flexible and scalable. Firmware development on the device end, it will be done in C++ with the Arduino framework or ESP-IDF which is common with ESP32 programming.

To supplement these, a React based user interface will be deployed, which will enable an administrator to monitor the status of devices, update, and update logs.

This study focuses on real world applications like a Smart Water Management System in Agriculture, where edge devices are both highly resource-constrained (low flash/RAM) and frequently solar powered as well as need periodically updating remote firmware. The proposed Lightweight secure OTA Core provides secure updates with signatures and authenticated encryption, delta updates to reduce network overheads and streamlined runtime and memory overheads through patch application by streaming or chunked updates and hardware accelerated cryptography. A combination of these properties enables large scale, periodic updates, that is, hundreds of distributed agricultural sensors are technically feasible and cost effective to update.

1.3 Background and Literature Survey

1.3.1 Overview of OTA Security in IoT

OTA update systems are now a standard feature in modern IoT deployments, yet securing them remains a complex task [3]. Best practice guidelines recommend encrypting firmware packages, authenticating update sources, and verifying integrity before installation. For example, Amazon FreeRTOS implements an OTA workflow where updates are cryptographically signed, downloaded via secure channels, and verified by a bootloader before activation [4]. Similarly, Tesla's automotive OTA infrastructure uses multi layered security, including code signing, encrypted transport, and staged rollouts, to ensure safety and prevent mass failures. These measures are crucial in mitigating risks associated with unauthorized firmware modifications and ensuring the reliability of IoT devices [5].

Espressif, the manufacturer of ESP32, provides an OTA API within its ESP-IDF framework, enabling secure firmware delivery over HTTPS [5]. However, while these existing solutions offer strong security, they often transfer the full firmware image, which can be inefficient for low bandwidth environments [6]. This inefficiency underscores the need for optimized update mechanisms that balance security with network resource constraints [7].

1.3.2 Delta Updates and Bandwidth Optimization

Delta updates reduce bandwidth usage by transmitting only the changes between firmware versions. This approach is particularly beneficial in large IoT deployments where simultaneous updates could congest networks. Industry research has shown that delta update mechanisms can reduce firmware transfer sizes by up to 90%. Google's Chrome OS update system and Espressif's experimental delta update implementations demonstrate the feasibility of this technique. Furthermore, delta updates help in minimizing network interruptions and reduce the likelihood of failed updates in unstable connections, which is critical for devices deployed in remote or resource constrained environments. For IoT, smaller update sizes also mean reduced energy consumption during wireless transmission, extending battery life for remote devices [8].

Additionally, this project explores lightweight NLP-assisted techniques to analyze firmware change patterns and update metadata to guide delta generation and patch segmentation, improving compression and reducing transmitted bytes.

1.3.3 Lightweight Cryptography for Constrained Devices

Resource constrained IoT devices require cryptographic algorithms that offer high security without excessive computational cost. ECC provides equivalent security to RSA with significantly smaller key sizes, making it ideal for embedded devices. AES-CCM is widely used in protocols such as LoRaWAN, providing both encryption and authentication in a single pass [9]. These lightweight algorithms are particularly advantageous for battery powered and low power IoT devices, as they minimize processing time and energy consumption while maintaining strong security guarantees. They also allow secure communication in networks with limited bandwidth and reduce latency in time sensitive applications. These algorithms have been proven to run efficiently on ESP32 class devices, especially when leveraging hardware acceleration [10].

1.3.4 Lightweight OTA Efficiency Enhancements

The recent studies show that updating OTA is not just important but also needs optimization so that it can be efficient across the resource constrained IoT based device. In addition to regular delta updates, there are newer approaches like smart patch segmentation and compression measures that are investigated to minimize the transmission overhead. The emergence of NLP methods to predict patterns of change in firmware and metadata used in the project is a new contribution toward analyzing patterns of firmware change and its metadata and guiding delta generation and enhancing patch efficiency[11]. Together with AES-CCM authenticated encryption and ECC-based signatures, such solutions can be used to improve the feasibility of secure OTA through reduced bandwidth use, memory footprint, and energy consumption. This is a point of emphasis of the Lightweight Secure OTA Core, as it strives to provide more security and high efficiency to ensure larger deployments of IoT can be made more sustainable[12].

1.3.5 Challenges and Limitations

Secure OTA updates for IoT devices face multiple challenges and constraints that make implementation complex. Designers must ensure that updates remain lightweight while providing strong security, taking into account real world conditions such as varying connectivity, energy limitations, and diverse deployment scenarios. Testing across different device types is also necessary to maintain reliability and robustness [13].

Key challenges in secure OTA for IoT:

- Limited device memory and processing capacity.
- Variability in network conditions.
- Maintaining security without excessive overhead.
- Handling heterogeneous hardware across an IoT fleet.

These limitations necessitate a carefully balanced solution that optimizes security and performance.

1.3.6 Conclusion

The literature demonstrates that while secure OTA mechanisms exist, few solutions effectively combine strong encryption, bandwidth efficient delta updates into a single, lightweight system. Many existing approaches focus on individual aspects of security or optimization, but do not address the combined requirements of constrained IoT devices in real world deployment scenarios. Integrating these elements holistically is critical for ensuring both security and reliability. This research addresses that gap by integrating these elements into a coherent framework for resource constrained IoT devices.

1.4 Research Area

1.4.1 IoT Security Engineering

The IoT Security Engineering is concerned with the design and implementation of security measures that could be used to safeguard connected devices and networks. It concerns the safety of the data transfer, the integrity of the firmware, and resistance to the cyber threats. Within the scope of this study, it addresses safe OTA updates systems, authentication processes, and to make sure that the IoT devices can safely update their firmware without disrupting their operational resilience or exposing confidential data to attackers.

1.4.2 Embedded Systems Development

Embedded Systems Development involves the development of software and hardware in resource constrained devices for example microcontrollers, and sensors. It incorporates low level programming, real time operating systems and optimization of the system to work efficiently. The study capitalizes on the embedded development of ESP32 platforms and combines it with OTA updates with a small footprint, processing optimization, memory and power consumption optimization, and system stability and responsiveness in real world IoT deployments.

1.4.3 Cryptography for Constrained Devices

Cryptography of constrained devices is the study of lightweight cryptography believed to be suitable to devices with limited memory, processing energy, and processing power. Elliptic Curve Cryptography and AES-CCM are techniques that allow safe communication as well as data integrity to be achieved without straining the apparatus. These algorithms are used in this research to protect OTA firmware updates, provide confidentiality, authenticity, and integrity without interrupting the low computational overhead of embedded IoT devices in diverse environments.

1.4.4 Wireless Communication Protocols

Wireless Communication Protocols focus on reliable and efficient data transmission between devices over wireless networks. It encompasses protocols like Wi Fi, BLE, LoRaWAN, and MQTT that regulate connectivity, data integrity and network efficiency. In this study, it is important to know and optimize wireless protocols to provide OTA updates, reduce latency, decrease bandwidth consumption, and maintain a consistent delivery of firmware even in limiting or unreliable network environments.

1.5 Significance of the Research

The work is significant because it offers a avenue of delivering safe, effective OTA upgrades to the low power IoT gadgets, making them last longer, be more secure, and be more maintainable. It is also concerned with efficiency in operations with delta updates and with cybersecurity with ECC and AES-CCM encryption, which do not compromise the performance to guarantee security. Besides this, the study highlights the need to utilize lightweight and scalable solutions that fit resource constrained settings. It can enable even low memory and battery powered devices to have secure updates since it reduces the computational load as well as consumes less network bandwidth without affecting their usual functioning.

In addition, the paper offers an integrated model that incorporates encryption, efficient update delivery, and resilience designs, rendering the study flexible to the heterogeneous IoT implementations, such as smart homes, industrial systems, and remote monitoring networks.

The study confirms the practical relevance of the framework in real life context taking into consideration different network characteristics, hardware constraints and issues of large scale deployment. It assists in dynamic planning of updates to make the most out of power usage, lessen upkeep, and limit downtime. The framework makes use of lightweight cryptography and delta updates but also increase the efficiency, reliability, and maintainability of the operation in a wide variety of IoT ecosystems. This makes it especially useful in sectors where failing of the devices or unauthorized access may cause severe economic, safety or operational consequences.

On the whole, the suggested framework fills the key gaps in current OTA solutions as it provides a holistic, practical, and secure solution ensuring a balance of performance, reliability, and high security assurances to various IoT devices.

2. RESEARCH GAP

Existing research on Over the Air updates often focuses either on security through methods such as signing and encrypted transport or on efficiency through techniques such as delta compression. However, these two aspects are rarely combined into a single, fully validated system designed for resource constrained microcontroller class devices.

The literature does not provide solutions that can simultaneously deliver signed and per device encrypted delta updates, where signatures are generated using ECC and encryption is performed with AES-CCM. This gap is particularly evident in devices with less than four megabytes of flash memory, where lightweight methods are required to ensure updates can still be securely applied under strict resource limits.

Furthermore, existing work lacks end to end empirical evaluations that measure the actual impact on CPU performance, RAM usage, and energy consumption during both patch generation and patch application. Such evaluations are especially important for ESP32-class hardware, where network conditions vary widely in terms of bandwidth, latency, and packet loss.

This project addresses these gaps by designing, implementing, and experimentally validating a complete Over the Air update pipeline. The proposed solution uses ECC based signing together with AES-CCM encryption for delta updates, operates on ESP32 hardware, and is evaluated with quantifiable targets for bandwidth reduction, memory and CPU efficiency, and update success rates under realistic network conditions. These techniques combine in a single framework, which not only makes the research efficient but also more practical and efficient compared to the previous OTA systems.

Assessment will be based on the following:

- Attaining high bandwidth savings of at least 70 % versus full image updates.
- Forcing low memory overhead when applying patches, less than 200 KB RAM usage.
- Have efficient cryptographic verification with signature verification time of not more than 80 milliseconds on ESP32 hardware.
- Has strong performance in lossy environments, where the success rate of updates in 2% loss networks is as high as 95%.

Area	Existing Research / Practice	Limitations / Gaps	Proposed Research Contribution
Secure OTA Updates in IoT Devices	Studies exist on using AES or ECC individually for securing firmware updates such as secure boot, encryption during transmission.	Most works focus on either security or efficiency, not both together few combine ECC for key exchange, with AES-CCM for authenticated encryption.	Implement a combined ECC and AES-CCM approach optimized for ESP32, ensuring both security and low computational overhead.
Delta Update Mechanisms	Delta update algorithms like bsdiff and xdelta3 are widely studied for desktop/server environments.	Limited research on applying delta updates to constrained IoT microcontrollers concerns about memory usage and patch reliability.	Integrate lightweight delta update mechanism for ESP32 firmware, reducing bandwidth and storage without breaking system stability.
Bandwidth & Energy Optimization	Research exists on compressing firmware or scheduling OTA updates to reduce network load.	Few studies quantify both bandwidth and energy savings in real ESP32 deployments under secure OTA constraints.	Measure and optimize bandwidth and energy consumption during secure OTA updates in real world IoT testbeds.
Existing OTA Frameworks for ESP32	ESP-IDF OTA API, Arduino OTA, and some custom solutions are available.	These lack built in advanced security (ECC and AES-aCCM) and bandwidth optimization with delta updates.	Extend existing ESP-IDF OTA framework to include integrated security and delta update logic, making it more production ready for secure IoT.

Table 2. Research Gap Table

3. RESEARCH PROBLEM

The research problem discussed in the current study is the difficulty of developing a lightweight and secure OTA update system to use in particular constrained IoT devices. Such devices frequently run in resource constrained environments where memory, processing power and battery capacity are highly limited and thus standard OTA solutions are inappropriate. Making sure that firmware updates are managed in a secure manner that does not overload the computer or the network with additional computational or network overhead is a vital prerequisite. Simultaneously, bandwidth consumption is to be minimized by ensuring an optimized update size, through delta updates which support low bandwidth networks and to save power.

The other significant issue of the problem is the provision of a dependable fail safe that will enable the devices to re boot safely in the event of an update failure to avoid bricking of devices or disruption of operation. Existing OTA systems will generally support one or two of these dimensions security, efficiency, or reliability but will seldom combine all three into one platform that can be used in heterogeneous IoT implementations. This leaves a gap in the application of practices to real world IoT settings where devices would be required to run continuously and safely under different network settings and hardware limitations.

The research problem is therefore the creation of a complete OTA framework that is strong on security, efficient on bandwidth and at the same time is lightweight enough to run on a low power and resource constrained IoT device. This issue is vital in the safe, reliable, and efficient implementation of the IoT systems in various uses.

3.1 Research questions

1. How can ECC, AES-CCM be efficiently implemented on ESP32 devices for OTA security?
2. What bandwidth savings can be achieved using delta updates compared to full firmware transfers?
3. How can NLP techniques be applied to analyze OTA update logs, metadata, or firmware diffs to improve update efficiency or classify update types?
4. How does network reliability and packet loss affect the performance and success rate of delta based OTA updates?

4. AIM AND OBJECTIVES

4.1 Main Objective

To design and implement a secure, efficient OTA update system for ESP32 based IoT devices that integrates ECC authentication, AES-CCM encryption, and delta update optimization.

4.2 Specific Objective

- Use ECC to sign and verify the firmware to achieve authenticity and integrity.
- Include AES-CCM encryption to secure update payloads to ensure confidentiality and anti tampering.
- Create server side delta algorithms like bsdiff and xdelta3 and build device side patching systems to reduce bandwidth consumption and update time.
- Explore NLP techniques for analyzing update logs or metadata to improve update management efficiency.
- Evaluate system performance in terms of
 - o Reduction of bandwidth (over full image updates).
 - o CPU and memory utilization on ESP32 devices.
 - o Energy consumption during update processes.
 - o Stability and update success under different network conditions.
- Ensure scalability of the OTA framework for heterogeneous IoT environments, from smart home appliances to large scale industrial sensor networks.
- Compare the proposed solution to the traditional OTA mechanisms in order to show the enhancement of security, efficiency, and robustness.
- Enhance the framework on the basis of cybersecurity best practices and standards to ensure long term device sustainability and compliance.
- Recommend how the OTA solution can be modified to suit the classes of IoT devices with different memory, CPU and connectivity limits.

ESP-IDF / Vendor OTA implementations.

Vendor SDKs such as ESP-IDF provide a robust starting point for device side update mechanics including HTTPS download. However, these implementations typically transfer full images and do not provide integrated signed and encrypted delta delivery they also often assume sufficient flash and RAM for streaming a whole image. SecureTrustOTA builds on the ESP-IDF boot and partition model but augments it with server side delta generation and patient, verified patch application to reduce transfer size and risk of bricking.

Cloud / Fleet update systems such as AWS FreeRTOS, Tesla.

Cloud scale solutions focus on rollout orchestration, telemetry, and staged deployment. These systems are mature at the orchestration level but assume non constrained endpoints or rely on cloud billing models. This project targets constrained on device processing and demonstrates practical bandwidth and energy benefits without depending on proprietary cloud services.

Delta algorithms and patching (bsdiff / xdelta3).

Delta tools are proven on servers and desktops, but naively applying them to microcontrollers can fail because patch application may exceed RAM or flash constraints. Prior works rarely quantify patch apply memory use on embedded devices. SecureTrustOTA adapts delta patching with chunked apply techniques, streaming decompression, and verified apply steps to keep RAM below target budgets.

5. RESEARCH METHODOLOGY

5.1 Overview of Methodology

Research methodology is based on a design science approach, which in this case involves the development of an artifact secure and bandwidth efficient OTA update system, to solve a real world problem. It uses a cyclical requirements gathering, system design, prototyping, testing, iteration process methodology. This would provide the assurance that the suggested OTA system does not only comply with the theoretical level security requirements, but it also works well in practice under the conditions of the IoT deployment.

The development process will be divided into four key phases:

1. Requirement Analysis & Technology Selection
2. System Design & Architecture Planning
3. Prototype Development & Integration
4. Testing, Evaluation, and Optimization

5.2 Requirement Analysis

The identification of the functional and non functional requirements of proposed OTA framework is the first stage. The functional requirements are the transfer of firmware is secure, there is delta update support in order to reduce the size of transmission. Non functional requirements include low CPU and memory overhead, low bandwidth consumption, quick update execution and network disruption resiliency, to maintain reliable operation in resource limited environments. Moreover, the requirements will be verified and focused considering the purpose of consultation with the guidelines of the IoT industry (ISO/IEC 30141:2018) and other standards related to security (NIST SP 800 183), as well as, the standards of best practices in the sphere of device security, firmware integrity, and efficient delivery of updates in the heterogeneous IoT networks.

5.3 Technology Selection

Careful selection of the technology stack is critical to ensure that the OTA framework remains secure, lightweight, and efficient while being compatible with constrained IoT devices. The chosen tools also support modular development, ease of integration, and scalability.

IoT Device Platform

- ESP32 due to integrated Wi Fi, hardware cryptography, and widespread adoption.

Firmware Development

- C++ via Arduino Core and ESP-IDF for embedded development flexibility.

Server Backend

- Python based Flask/FastAPI running on a VPS, responsible for firmware patch generation, encryption, and device communication.

Web Interface

- React for device management dashboard and OTA monitoring.

Encryption Algorithms

- ECC for signing and AES-CCM for encryption.

Delta Update Tool

- bsdiff or xdelta3 for binary patch generation.

5.4 System Diagram

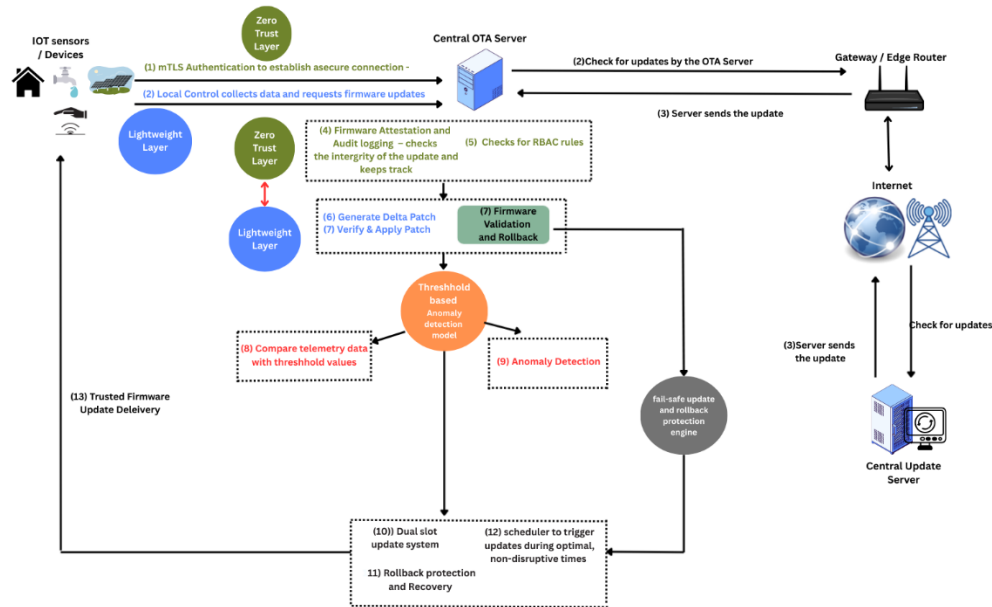


Figure 1: System diagram

The system starts with the IoT devices such as ESP32 based sensors which constantly gather data and execute the local control functions. A secure connection is established between the device and the backend by mTLS authentication when a firmware update is required. This makes sure that the server and the device can trust one another prior to any sensitive information being passed across. After authentication, the device then asks the OTA server to fetch it updates and the OTA server then connects with the central update server via the internet to get the most recent firmware versions.

Thence, the update process is well designed. Zero Trust Model makes sure that all updates to the firmware are attested and recorded, and RBAC authorization is performed to ensure that the update is only applied to authorized devices. The system transmits a lightweight model instead of sending complete firmware files. Delta patches containing only the differences between the old and new firmware are generated by the system. These patches are then checked, installed, and reverted to a safe version when needed.

Another layer of security is also provided by a Threshold Based Anomaly Detection Model that observes telemetry data. Should there be some suspicious activity, the update procedure can be stopped or tagged to be reviewed, so the malicious firmware is not able to attack the devices. In conjunction with this, the Fail Safe and Rollback Protection Engine comes with a dual slot system inspired by which devices should not be affected by a mishap and that they will always be operational.

The backend OTA management system and the ESP32 device side process. A back end implemented in Python frameworks such as Flask/Django receives update requests and determines the version databases, as well as creates delta patches with tools such as bsdiff or xdelta3. Security is ensured by means of strong encryption with ECC signatures and AES-CCM encryption, by ensuring that the updates cannot be altered. The ESP32 client, compiled with the Arduino SDK and C++, downloads, verifies, and executes the delta patch on the device side using ESP-IDF OTA APIs. If the update is valid, the firmware is updated, and if it is not, the system restores the working version.

5.5 Component Diagram

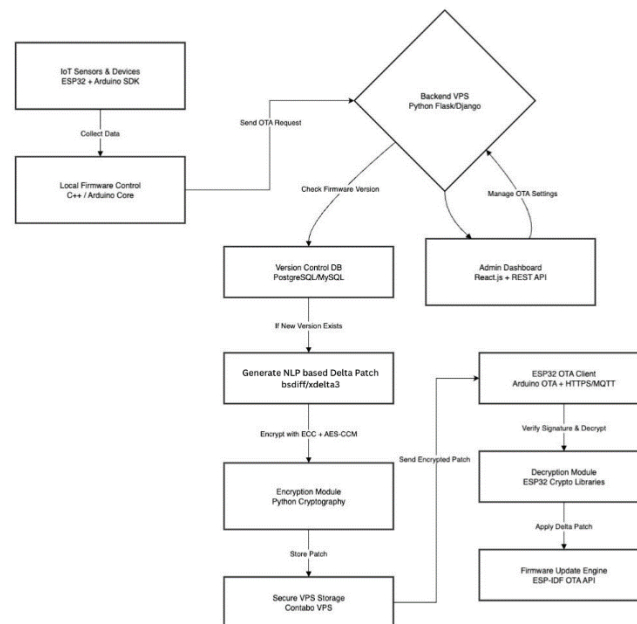


Figure 2: Component diagram

This diagram describes the secure Over The Air firmware update workflow on ESP32 based IoT devices. Initially, the IoT devices with ESP32 and configured through the Arduino SDK peripherals are responsible for data harvesting and firmware based local control. These devices run under the control of locally deployed firmware for perpetual operations, written in C++. However, during the periods in which updated firmware can be released, the devices initiate backend communications and present an OTA request.

Such requests are serviced by backend virtual private servers built on frameworks like Python Flask, Django, etc. Serving as the core controller, the backend checks the version control database, which is usually housed on PostgreSQL or MySQL, and determines if the device in question is up to date with the firmware or needs to be updated. When an update is required, the backend does not just serve the entire firmware, rather, it generates a delta patch with bsdiff or xdelta3. This patch contains the changes between the current and the new version, which substantially reduces the update size and saves bandwidth.

The system uses delta patching techniques that are enhanced by NLP to maximize this process

further. The backend uses natural language processing models to semantically analyze semantic analysis of firmware source code or configuration scripts (instead of applying binary diff algorithms) and then compiles. With greater insight into similarities and differences in structures and functions, the NLP based pipeline can find significant transformations of the code, unnecessary changes and more intelligently compress the update instructions. This hybrid technique minimizes patch size over conventional binary diffing as well as enhances consistency in instances where the firmware versions are not similar.

Once the delta patch is generated, it goes through a security protocol. The patch is processed through a protective veil using Python's cryptography libraries. Here, a blend of elliptic curve cryptography for digital signatures and AES-CCM is used for encryption to secure the patch data. This method guarantees that the update is only accessible to authorized devices and ensures that no unauthorized entity can interfere with the patch during its transfer. The patch is then encrypted, and it is subsequently stored on the VPS until it is needed for delivery.

The encrypted patch is retrieved by the ESP32 OTA client on the device, who communicates through HTTPS, MQTT, and similar protocols. The client holds cryptographic libraries and checks the patch before any decryption process. If the patch is encrypted with a proper signature, it is safe to proceed to the decryption stage. With the ESP-IDF OTA API, the device applies the delta update, integrating the patch with the existing firmware to construct the new version on the device. This approach is optimal for resource limited IoT devices.

After pushing the update, there's this firmware validation phase that kicks in. The device runs checks on the new firmware's integrity using checksums and verifies the ECC signature. If everything lines up clean, it switches over as the active version running things now. But if things go sideways here due to corrupted files or tampering attempts, the system bounces back to the last stable firmware automatically. Keeps things running without turning into a paperweight.

Over on the admin side, they're handling all this through a React.js dashboard hooked into REST APIs or whatever. Lets them tweak OTA settings, watch device statuses in real time, force updates when needed. By mixing delta updates that only send changed bits with solid crypto and rollback safeguards, you get a system that keeps IoT gear patched securely out in the wild. No bricked devices. No truck rolls required. Just a solid way to maintain stuff remotely without ever touching the physical hardware directly.

5.6 Dataset and Testing Environment

Although OTA systems do not typically require a traditional dataset, testing will involve a synthetic firmware update dataset consisting of multiple firmware builds with controlled variations. This enables precise measurement of delta update efficiency. Network simulation tools like NetEm on Linux will be used to emulate real world conditions such as latency, packet loss, and bandwidth limitations.

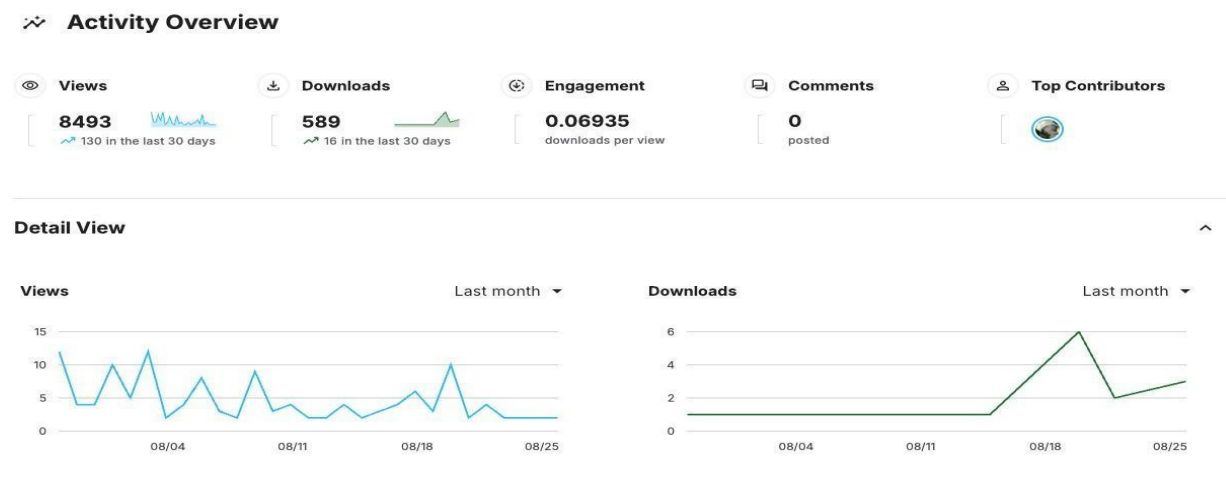


Figure 3: Datasets

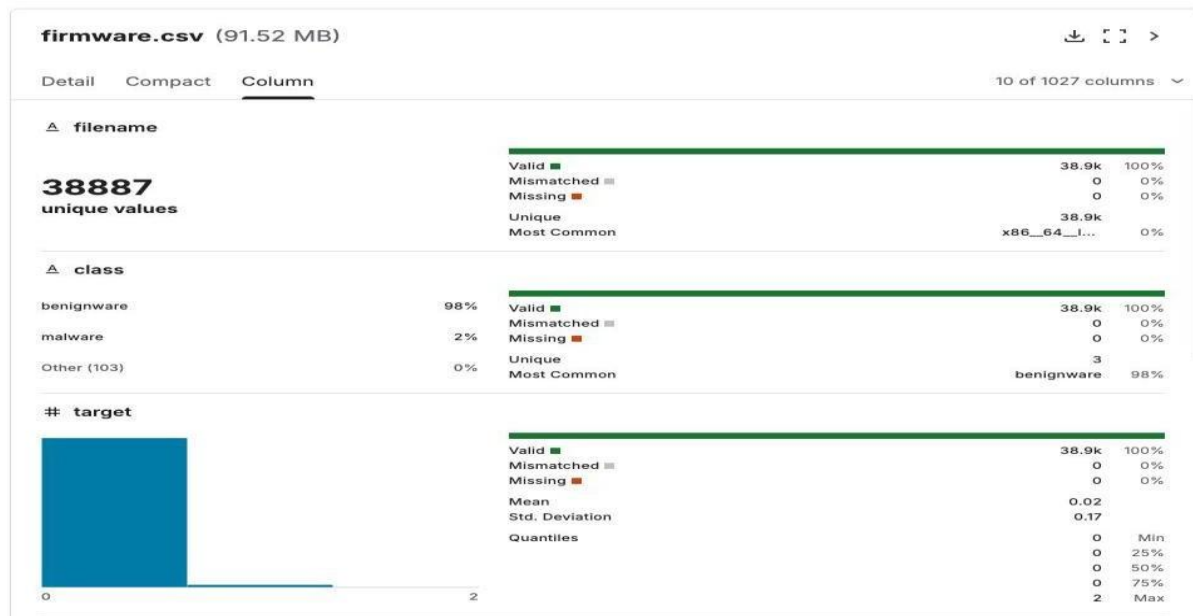


Figure 4: Datasets

Link to Dataset: <https://www.kaggle.com/datasets/datamunge/iot-firmware-image-classification>

Activity Overview



Detail View

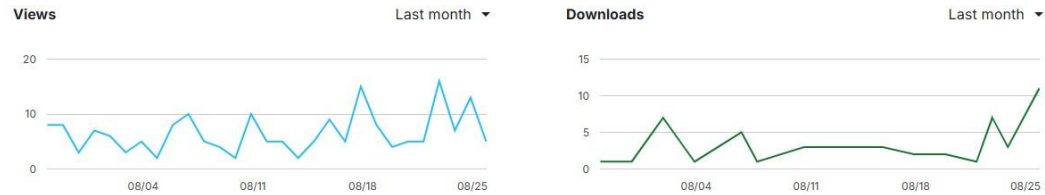


Figure 5: Datasets

Link to Dataset: <https://www.kaggle.com/datasets/mdabdulalemo/cic-iot-dataset2023-updated-2024-10-08>

Hyperlocal Urban Environmental Monitoring Dataset

Data Card Code (0) Discussion (0) Suggestions (0)

1 <> Code Download



Detail View



Figure 6: Datasets

Link to Dataset: <https://www.kaggle.com/datasets/avitarus/hyperlocal-urban-environmental-monitoring-dataset>

5.7 Development Flow

The development of the SecureTrustOTA framework follows a structured workflow to ensure reliability, security, and efficiency at each stage. Each step is designed to handle firmware preparation, secure transmission, and safe update application for constrained IoT devices.

5.8 Evaluation Plan

Evaluation of SecureTrustOTA is conducted through two classes of experiments:

1. Microbenchmarks focus on measuring cryptographic and patch application costs on a single ESP32 device.
2. System benchmarks focus on fleet update performance, reliability, and space and energy trade offs under controlled network conditions.

5.8.1 Baselines

- Native ESP-IDF full image OTA over HTTPS, with no delta, signed but not encrypted payload
- Unoptimized xdelta3 server to client workflow

5.8.2 Independent Variables

- Firmware delta size: small, medium, large
- Network bandwidth: 56 kb/s, 512 kb/s, 5 Mb/s
- Packet loss: 0 %, 1 %, 2 %, 5 %
- Concurrency: 1, 10, 100 devices

5.8.3 Dependent Variables and Metrics

- Patch transfer size in bytes
- Total update time in seconds
- Peak RAM usage in kilobytes
- CPU utilization in percentage per core
- Energy consumed during update in millijoules, measured with USB power meter
- Signature verification time in milliseconds

- Update success rate in percentage

5.8.3 Equipment and Tools

- ESP32 development boards
- VPS for server hosting
- NetEm for network emulation of latency, bandwidth, and packet loss
- USB power meter or Monsoon device for energy measurement
- Wireshark and tcpdump for network traces
- pytest and ArduinoUnit for automated test scenarios

5.8.4 Acceptance Criteria

- Achieve at least 70 % average bandwidth reduction compared to full image OTA
- Peak RAM usage of 200 kilobytes or less while applying patches
- Update success rate of at least 95 % under 2 % packet loss

5.8.5 Experimental Steps

The following table outlines the experimental steps along with the expected outputs and measurements for each step.

Step	Description	Expected Output / Measurement
1	Apply microbenchmark for cryptography and patch application on a single ESP32	Time (ms), CPU%, RAM usage, energy consumed
2	Transfer delta vs full image OTA under 56 kb/s	Patch transfer size, total update time
3	Transfer delta vs full image OTA under 512 kb/s	Patch transfer size, total update time
4	Transfer delta vs full image OTA under 5 Mb/s	Patch transfer size, total update time
5	Evaluate patch success under 0%, 1%, 2%, 5% packet loss	Update success rate
6	System level concurrency test (1, 10, 100 devices)	CPU%, RAM usage, energy consumption, update time

Table 3. Experimental Steps

6. SELECTION OF TOOLS AND TECHNOLOGY

6.1 Technologies

The technology selection to adopt to secure lightweight OTA system should be able to compromise between security, efficiency and compatibility with resource constrained devices.

ESP32 Microcontroller has been chosen as the main IoT device because it has an inbuilt Wi Fi, a dual core processor, a hardware accelerated cryptography, and a wide community support. Its support of both the Arduino framework and Espressif IoT Development Framework lets it be flexibly programmed and combine with advanced OTA capabilities.

Python is used for backend developing in the VPS to handle things like firmware storage, delta patch creation, AES-CCM encryption, ECC signature creation and communication with devices. Lightweight and scalable API frameworks like Flask or FastAPI can be used in the IoT. The python ecosystem consisting of libraries such as PyCryptodome to carry out cryptography makes it the best to develop secure and reliable server side code. A user friendly dashboard is built with React and enables administrators to track device status, invoke OTA updates, and visualize update progress. The component based architecture of React permits modularity in the design of the UI which is easy to expand with functionality as the system grows.

The OTA backend can be hosted in VPS which offers a dedicated and reliable server environment. In comparison to cloud services where software deployment, file storage, and security settings are more complex to integrate, a VPS is capable of full control over all these aspects, and is affordable enough to be used in academic research.

Digital signatures and key exchange are supported via Elliptic Curve Cryptography, the digital signature and key exchange can be used to ensure a high level of security with a small key size, and authenticated encryption with AES-CCM, authenticated encryption is a combination of confidentiality and integrity checking. These options make sure that the update of firmware is safe without the excessive consumption of the resources of the device.

Binary patches are created using Delta Update Algorithms like bsdiff or xdelta3, and only the difference between versions of the firmware is sent. This plays a major role in decreasing network loads and energy use on low power IoT devices.

6.2 Tools

The following tools and libraries are recommended to implement and test the system:

Category	Tool/Library	Description
Firmware Development Tools	Arduino IDE/ PlatformIO	Simplifies ESP32 programming, library management, and OTA client integration
	ESP-IDF	Provides native support for advanced ESP32 features, including secure OTA APIs, hardware accelerated AES, and ECC libraries
Cryptography Libraries	ArduinoECCX08	Supports ECC key generation and ECDSA signature verification on Arduino and ESP32 devices.
	MbedTLS	Embedded cryptography library compatible with ESP IDF, supporting AES-CCM and ECC.
	PyCryptodome (Python)	Provides server side AES CCM encryption, ECC signing, and hashing operations.
Delta Update Tools	bsdiff / bspatch	For generating and applying binary patches.
	xdelta3	Alternative tool for binary diff generation with high compression efficiency.
Server, Deployment Tools	Python 3.x	Backend logic and patch generation.
	Flask / FastAPI	Lightweight API frameworks for handling OTA requests.
	Linux VPS	Provides a secure, persistent environment for the OTA server
Monitoring, Testing Tools	MQTT.fx / Mosquitto	For simulating IoT communication over MQTT.
	Wireshark / tcpdump	For monitoring network traffic and verifying encryption.
	Unit Testing Frameworks	pytest for Python backend, ArduinoUnit for device side testing.
	Network Simulation Tools	NetEm on Linux to emulate bandwidth constraints, packet loss, and latency.

Version,Control CI/CD	Git / GitHub	Source code management, collaboration, and version tracking.
	GitHub Actions / Travis CI	Automates firmware builds, delta patch generation, and test execution.

Table 4. Selected tools and descriptions

By combining these technologies and tools, the system can achieve a robust, efficient, and secure OTA update pipeline for IoT devices, while remaining practical for deployment and testing within academic research. The selection balances lightweight device execution, strong cryptographic security, and bandwidth efficient update delivery.

7. PROJECT REQUIREMENTS

7.1 Functional Requirements

Secure Firmware Delivery

- Provide the firmware updates through VPS server to ESP32 devices in which the updates are cryptographically signed using ECC and encrypted with AES-CCM to avoid tampering and unauthorized access.

Delta Update Mechanism

- Introduce delta update mechanism, so ESP32 devices can update on differences between the version of the firmware, saving on data transmission and improving performance of devices with low bandwidth.

Verified patch application

- Ensure that the ESP32 client only applies a firmware update after verifying the ECC signature and AES-CCM integrity, and apply patches piece by piece to stay within RAM limits and prevent device crashes.

Lightweight Cryptographic Operations

- Use lightweight ECC and lightweight AES-CCM implementations that are small and lightweight to run on resource constrained IoT devices and that do not overload processing power, memory, or battery life.

7.2 Non Functional Requirements

- The typical incremental update average size for bandwidth is $\leq 30\%$ of the full image, with a target reduction of $\geq 70\%$.
- The peak RAM required for applying a patch in memory is ≤ 200 KB.
- The CPU time for signature verification and decryption per patch is ≤ 120 ms on an ESP32

microcontroller running at 240 MHz.

- The reliability of the update is measured by a success rate of $\geq 95\%$ under 2% packet loss and $\geq 90\%$ under 5% packet loss.
- The energy consumed per successful patch application is $\leq X$ mJ, this should be measured, reported in the thesis, and compared to a baseline from measurements.
- The security requirement mandates a false acceptance rate of 0 for unauthorized updates, with ECDSA and AES-CCM verification being mandatory.

7.3 User Requirements

Dashboard access

- User able to monitor devices, initiate updates, and view logs through a React based dashboard

Update feedback

- User receives clear status messages for updates, including success, failure, or rollback events

Efficiency

- User is ensured that updates are applied quickly and efficiently, minimizing bandwidth use and RAM/CPU load on devices.

Scalability

- User can manage multiple devices efficiently, with bulk update support and scalable performance

7.4 System Requirements

Microcontroller

- ESP32 with Wi Fi connectivity and at least 4MB flash memory to support dual bank firmware

Server

- VPS running Linux, Python 3.x, Flask or FastAPI, with sufficient storage for firmware repositories

Development Environment

- Arduino IDE or PlatformIO for ESP32 firmware, React for frontend, and Python libraries for server side cryptography and delta updates

Networking

- Secure communication between server and devices via HTTPS or MQTT, supporting intermittent connectivity and automatic retries

Testing & Monitoring

- Tools such as NetEm for network simulation, Wireshark or tcpdump for traffic inspection, and logging dashboards for event tracking.

8. BUDGET AND BUDGET JUSTIFICATION

The estimated budget for this research project has been prepared to account for hardware acquisition, software development tools, server hosting, and miscellaneous expenses. The budget is designed to be realistic, reflecting costs for academic level IoT development while ensuring the research objectives are achievable.

8.1 Total Estimated Budget

Item	Quantity	Unit Cost (LKR)	Total Cost (LKR)	Justification
Laptop/PC (Already existing)			0	Already available personal resources.
IoT Development Boards (ESP32 / STM32)	4	6,500	26,000	ESP32 boards are cost effective and widely used in IoT security research.
Secure Boot/Encryption Modules (ATECC608A Crypto chip)	2	3,500	7,000	Used to simulate hardware based cryptographic security and support ECC features.
Storage Devices (USB drives / SD cards)	4	1,000	4,000	For transfer firmware, test data storage, and recovery testing.
Networking Devices (Wi Fi Router / Switch)	1	8,500	8,500	Required to simulate enterprise like IoT device connectivity for OTA delivery.

Software Tools & Libraries			0	Most of the development tools (Arduino IDE, PlatformIO, OpenSSL, Wireshark, etc.) are open source.
Cloud Resources (Testing – Free Tier)			0	Cloud services such as AWS IoT Core / Azure IoT Hub will be used within free student credits or free tier.
Printing & Documentation			3,000	Printing of project proposal, final report, and Other essential materials.
Cables, Connectors, Power supplies			5,000	To power up and connecting development boards, routers, and test setups.

Table 5. Budget Summary Table

Total Estimated Cost: LKR 53,500

Budget Justification

The budget proposed is carefully planned to ensure the implementation (cost will not be too high compared to the Sri Lankan academic environment).

- **Hardware Boards:** aside from the affordability of the ESP32/STM32 boards, they are powerful enough to simulate actual IoT devices so that OTA updates can be tested. Each of the members will need its own independent development and test boards.
- **Crypto Modules:** ATECC608A secure elements have been included to represent a small set of the crypto modules, this allows hardware based cryptography to be realistically illustrated without excess units.
- **Networking:** A common Wi Fi router would provide a shared testbed on which all OTA update experiments would be executed in conditions that reflect practical IoT deployment situations.

- Software Tools: Development, and cryptographic libraries are free/open source and thus, costs are reduced to a minimum.
- Cloud Services: Student plans and free options available with compute services will be used to simulate an OTA delivery environment free of subscription cost.
- Printing & Miscellaneous: Required in the creation of printable reports or documentation as well as smaller project installation requirements.

The budget justifies all expenditures as essential for the successful completion of the research. Hardware and VPS are necessary to implement and evaluate the OTA system in realistic conditions, while software and open source tools minimize costs and ensure reproducibility.

9. GANTT CHART

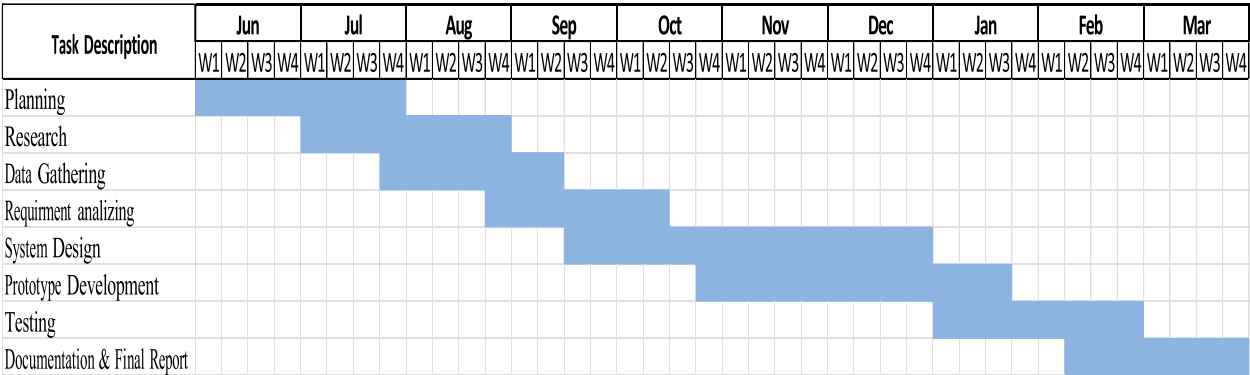


Figure 7: Gantt Chart

10. COMMERCIALIZATION

Smart water management system for Agriculture

One practical application of this research is in Smart Water Management Systems for Agriculture. In such deployments, large numbers of IoT devices are installed in fields to monitor and manage water usage. These devices are resource-constrained, often solar powered, and need regular firmware updates to stay secure and functional.

Commercial Value and Benefits:

- **Safe upgrades:** ECC signatures and AES-CCM encryptions are used to guarantee integrity of the firmware and resist modifications.
- **Efficient updates:** Delta updates transmit only difference so that the cost of data transfer is reduced, and bandwidth is saved.
- **Lightweight implementation:** Is developed to support devices with a small amount of memory and energy, to reduce the processing load and power usage.
- **Scalable:** Enables one to update hundreds of distributed sensors reliably and quickly.
- **Practical impact:** Enables sustainable farming by ensuring the reliability of the IoT system, minimizing the maintenance effort, and increasing the life cycle of the devices.

Smart Homes & Consumer Devices: Everyday IoT devices, such as smart plugs, cameras, and appliances, benefit from seamless updates that guarantee device safety. Customers can confidently update their devices without the risk of malfunctions, improving user trust and satisfaction.

Smart Metering & Energy Grids: Utilities can deploy millions of smart meters with confidence, knowing that updates will not disrupt billing or regulatory compliance. Our framework ensures continuous operation and minimizes the need for costly technician interventions, helping maintain smooth energy distribution and accurate data collection.

Simple ROI Example

- Fleet size: 10,000 devices
- Full image size: 1.0 MB, Average delta per update: 200 KB (80% savings)
- Bandwidth cost: LKR 6.04 per MB, Monthly patching: 10% of devices
- Full image approach cost: $1,000 \text{ devices} \times 1.0 \text{ MB} \times \text{LKR } 6.04 = \text{LKR } 6,040.00 / \text{month}$
- SecureTrustOTA delta cost: $1,000 \text{ devices} \times 0.2 \text{ MB} \times \$0.02 = \text{LKR } 1,208.00 / \text{month}$

Monthly Savings: LKR 4832.00 for this illustrative fleet. Savings scale linearly with fleet size and update frequency.

11. CONCLUSION

The study provided in this paper is about the design of Lightweight Secure OTA Core in resource constrained IoT devices, in this case ESP32 based microcontrollers. The proposed system solves the key challenges of safe and effective OTA updates in low power settings by enabling Elliptic Curve Cryptography as the method to perform firmware authentication, AES-CCM as the method to encrypt payload, and delta update as the method to optimize bandwidth.

The study, through organized methodology, including requirement analysis, system design, modular development and iterative testing, shows that it is not impossible to achieve a high level of security and at the same time to reduce the consumption of devices and network resources. Fail safe mechanisms will also provide system reliability, hence failure of the device during an update operation.

Moreover, using Python, Arduino/ESP-IDF, React, and VPS infrastructure, the study demonstrates how to practically and scalably implement OTA updates that are applicable to real world IoT applications. The increased Gantt chart and project requirements give a good roadmap of the development, testing and evaluation, which is why this research is not only theoretically important, but also extremely applicable in practice.

The work is an addition to the academic and practical knowledge of the lightweight, secure OTA systems, where a significant knowledge gap existed in literature, where the security, efficiency, or fail safety measures used to be held back. The results of the research are supposed to inform future IoT implementations, especially in an area where network and device constraints are significant.

REFERENCES

- [1] S. Singleton, "OTA for IoT: What It Is, How It Works, and Why It Matters," 1 April 2025. [Online]. Available: <https://memfault.com/blog/ota-for-iot/>. [Accessed 29 August 2025].
- [2] S. E. Jaouhari, "Toward a Secure Firmware OTA Updates for Constrained IoT Devices," 2022. [Online]. Available: <https://doi.org/10.1109/ISC255366.2022.9922087>. [Accessed 2025].
- [3] "Secure firmware OTA updates for constrained IoT devices," [Online]. Available: <https://doi.org/10.1109/ISC255366.2022.9922087>. [Accessed 2025].
- [4] "The role of OTA updates in Industrial Connectivity: Essential or Optional?," [Online]. Available: <https://www.exorint.com/exor-innovation-blog/the-role-of-ota-updates-in-industrial-connectivity-essential-or-optional>. [Accessed 29 August 2025].
- [5] "Over The Air Updates (OTA)," 2016 - 2025. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/ota.html>. [Accessed 29 August 2025].
- [6] "Secure Boot and Firmware Integrity in Industrial IoT Devices," 18 August 2025. [Online]. Available: <https://promwad.com/news/secure-boot-firmware-integrity-industrial-iot>. [Accessed 29 August 2025].
- [7] E. Systems, "Espressif Systems," 2025. [Online]. Available: https://www.espressif.com/documentation/esp32_datasheet_en.pdf. [Accessed 29 August 2025].
- [8] B. Wald, "Very Technology," [Online]. Available: <https://www.verytechnology.com/insights/delta-updates>. [Accessed 29 August 2025].
- [9] G. T. A. De Simone, "Incremental Firmware Update Over-the-Air for Low-Power IoT Devices over LoRaWAN," [Online]. Available: <https://arxiv.org/html/2505.13764v2>. [Accessed August 2025].
- [10] S. M. Y. A. G. M. S. Z. Vahdati, "Comparison of ECC and RSA Algorithms in IoT Devices," 2019. [Online]. Available: <https://www.jatit.org/volumes/Vol97No16/6Vol97No16.pdf>. [Accessed 2025].
- [11] J. K. S. U. C. I. Sci, "Incremental Firmware Updates for Low-Power IoT Devices over LoRaWAN," [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S1474034621002676>. [Accessed August 2025].
- [12] L. T. P. H. T. N. M. T. Nguyen, "Over-the-Air Programming Techniques for Low-Power IoT Devices," 2022. [Online]. Available: <https://doi.org/10.1109/ACCESS.2022.10480848>. [Accessed August 2025].
- [13] S. Kolli, "Empowering the future of embedded firmware with OTA systems," 2025. [Online]. Available: <https://blogs.infoservices.com/embedded-system/>. [Accessed August 2025].
- [14] E. Bouvet, "Secure firmware Over-The-Air updates for IoT: Survey, challenges and solutions," 2022. [Online]. Available: <https://doi.org/10.32604/cmt.2022.019302>. [Accessed August 2025].
- [15] M. D. (NIST), "Recommendation for Block Cipher Modes of Operation: the CCM Mode for Authentication and Confidentiality," 2007. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-38C>. [Accessed 2025].